IOT-ENABLED SMART BUS ROUTE MONITORING AND REVENUE MANAGEMENT SYSTEM

INDIVIDUAL THESIS

Aththanayaka N. D.

(IT22925268)

Individual component: IoT-Integrated Driver Monitoring and Operational Reporting Mobile Application

Group ID: R26-ISE-004

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specializing in Information Systems Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Aththanayaka N. D.

Student ID: IT22925268

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Sanjeeva Perera		
Co-supervisor: Buddima Attanayaka		

ABSTRACT

Public bus operations in Sri Lanka continue to experience weak real-time operational communication and limited visibility of driver behaviour during active service. Existing bus monitoring solutions commonly prioritise vehicle location, passenger information, or isolated telematics functions, while providing limited support for an integrated driver-facing and owner-facing mobile workflow. This individual research component designs and evaluates an IoT-integrated mobile application for operational reporting and real-time driver behaviour monitoring within a wider smart bus route monitoring and revenue management system. The solution combines onboard telemetry from an ESP32, NEO-6M GPS module, and MPU6050 inertial measurement unit with secure cloud communication, Spring Boot backend services, PostgreSQL storage, and a React Native mobile application. The driver can update bus availability, start and end trips, report breakdowns, and receive immediate warnings. The bus owner can review operational summaries, driver behaviour events, schedules, earnings, and route-related payment information supplied by the integrated platform. A rule-based telematics mechanism detects overspeeding, aggressive braking, aggressive turning, and abnormal slow movement from GPS speed, acceleration, gyroscope, location, and timestamp data. In the integrated project evaluation, 120 controlled driving trials produced an overall driver-event detection accuracy of 92.6%, with 93.3% for overspeeding, 91.6% for aggressive braking, 91.8% for aggressive turning, and 93.6% for abnormal slow movement. The results demonstrate that a low-cost IoT and mobile architecture can provide timely safety feedback while improving operational visibility for bus drivers and owners. The individual contribution is the integration of mobile operational reporting, real-time driver alerts, and owner-side visibility into the smart bus management ecosystem.

Keywords: Driver Monitoring, Mobile Application, IoT Sensors, Bus Operations, Driver Telematics

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Mr. Sanjeeva Perera, and co-supervisor, Mr. Buddima Attanayaka, for their guidance, constructive feedback, and continuous support throughout this research. I also appreciate the academic staff of the Faculty of Computing at the Sri Lanka Institute of Information Technology for providing the academic environment and technical guidance required for the project.

I am grateful to the System Administrator and officers of the Road Passenger Transport Authority for sharing practical insights on current bus operations, schedules, operational difficulties, and stakeholder requirements. I also thank my research group members for their collaboration in integrating the IoT hardware, scheduling, route monitoring, revenue, backend, web, and mobile components. Finally, I thank my family and friends for their encouragement and support throughout the research period.

TABLE OF CONTENTS

Right-click and update field to generate the Table of Contents.

LIST OF FIGURES

Figure 4.1 Component architecture within the complete solution

Figure 4.2 Real-time driver behaviour detection and alert workflow

Figure 5.1 Driver-side operational workflow

LIST OF TABLES

Table 1.1 Individual component scope

Table 2.1 Literature gap synthesis

Table 3.1 Data items and sources

Table 3.2 Evaluation measures

Table 4.1 Functional requirements

Table 4.2 Non-functional requirements

Table 4.3 Technology stack and justification

Table 5.1 Driver behaviour rule configuration

Table 5.2 Functional verification scenarios

Table 6.1 Driver behaviour detection results

Table 6.2 Results mapped to individual objectives

Table 7.1 Comparison with prior approaches

Table 8.1 Commercialisation model

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
DSR	Design Science Research
ESP32	Espressif 32-bit Microcontroller
GPS	Global Positioning System
IMU	Inertial Measurement Unit
IoT	Internet of Things
ITS	Intelligent Transportation System
JWT	JSON Web Token
MQTT	Message Queuing Telemetry Transport
RPTA	Road Passenger Transport Authority
REST	Representational State Transfer
TLS	Transport Layer Security
UI	User Interface

1 INTRODUCTION

1.1 Overall project context

Public bus transport is a critical mobility service in Sri Lanka, but daily operations are affected by unreliable service availability, manual operational processes, route deviations, scheduling disruptions, limited driver-safety visibility, and weak links between real-world bus operation and management decisions. The overall group project, titled "IoT-Enabled Smart Bus Route Monitoring and Revenue Management System," addresses these limitations through an integrated Internet of Things (IoT) architecture that combines onboard telemetry, route monitoring, policy-driven scheduling, distance-based revenue management, and stakeholder applications.

The wider system uses an onboard ESP32 microcontroller with GPS and inertial sensing to capture vehicle movement. Data is transmitted to cloud and backend services where different group components process route compliance, scheduling rules, revenue calculations, and driver behaviour. Web and mobile applications then expose relevant information to transport authorities, bus owners, and drivers. This individual thesis focuses specifically on the driver/owner mobile application and the real-time driver behaviour monitoring capability.

1.2 Individual component context

The individual component is titled "IoT-Integrated Driver Monitoring and Operational Reporting Mobile Application." Its primary user is the bus driver and its secondary user is the bus owner. The mobile application is intended to provide a practical digital interface for daily bus operations while also delivering immediate safety feedback based on onboard IoT sensor data.

During service, the driver can authenticate, view the assigned bus and route, update whether the bus is available, start or end a trip, update trip status, and report a breakdown. In parallel, the onboard device produces location, speed, acceleration, gyroscope, and timestamp data. These measurements are analysed to detect overspeeding, aggressive braking, aggressive turning, and abnormal slow movement.

When an event is detected, the system generates a mobile warning for the driver and stores the event for later owner-side review. The owner interface provides trip summaries, driver behaviour reports, schedule information, earnings, and route-related payment information received from the broader integrated system.

1.3 Problem statement

Existing bus operations provide limited digital support for combining operational reporting with real-time driver behaviour monitoring. Drivers may have no single mobile interface to report bus availability, trip progress, or breakdown events, while bus owners have limited access to consolidated trip and driver-safety information. At the same time, unsafe manoeuvres such as overspeeding, aggressive braking, aggressive turning, and abnormal slow movement may occur without immediate feedback to the driver.

This separation between operational communication and safety monitoring reduces situational awareness and delays operational response. It also limits the ability of bus owners to understand how a vehicle was operated during a trip. A practical solution therefore needs to integrate real-time IoT telemetry, mobile alerts, driver operational reporting, and owner-side visibility within a single component that can connect to the larger smart bus management platform.

1.4 Research gap

Prior Intelligent Transportation System research has established GPS-based tracking, passenger information systems, and sensor-based driver telematics. However, these capabilities are often implemented as separate products or research prototypes. GPS bus systems typically emphasise location and arrival information, while driver telematics research often evaluates driving events or driver profiles without integrating the results into a bus-specific operational mobile workflow [2]–[6].

The gap addressed by this individual component is the limited integration of three functions: (1) IoT-based real-time driver behaviour detection, (2) immediate mobile feedback to the driver, and (3) operational and owner-side reporting for bus

services. The proposed component combines these functions in a single mobile application connected to the shared IoT and backend platform.

1.5 Aim and objectives

The aim of this research is to design, implement, and evaluate an IoT-integrated mobile application that supports real-time driver behaviour monitoring, operational reporting, and owner-side visibility within a smart bus management system.

The specific objectives are:

- Develop a mobile application for bus drivers and owners to support authentication, bus availability, trip status updates, breakdown reporting, and operational summaries.
- Integrate onboard IoT data including GPS location, vehicle speed, acceleration, gyroscope readings, and timestamps with backend processing and the mobile application.
- Develop a rule-based driver behaviour detection mechanism for overspeeding, aggressive braking, aggressive turning, and abnormal slow movement.
- Generate real-time driver alerts and retain detected events for owner-side monitoring and reporting.
- Evaluate the detection accuracy, alert flow, data transmission reliability, and operational reporting functions of the component.

1.6 Research questions or hypotheses

- RQ1: How reliably can low-cost GPS and IMU telemetry identify unsafe or inefficient bus-driving events in real time?
- RQ2: How can detected driving events be converted into timely and understandable mobile alerts for bus drivers?
- RQ3: How can one mobile application support both driver-side operational reporting and owner-side operational visibility without duplicating the functions of the authority web system?
- RQ4: How effectively can the mobile component integrate with the shared IoT, backend, scheduling, and revenue modules of the complete smart bus system?

1.7 Scope limitations and assumptions

Table 1.1 Individual component scope

In scope	Out of scope

In scope	Out of scope
Driver and owner mobile interfaces; bus availability; trip status; breakdown reporting; real-time driver alerts; trip summaries; behaviour reports; schedule, earnings and payment visibility; integration with IoT telemetry and backend APIs.	Passenger ticketing and automated fare collection; authority web-admin functions; route map-matching algorithm ownership; dynamic scheduling algorithm ownership; distance-based revenue formula ownership; in-cabin video/drowsiness detection; rail or multimodal integration.

The research assumes that the shared onboard IoT device and backend services are available to supply validated telemetry and integrated data. Driver behaviour detection is implemented as a rule/threshold-based mechanism rather than as a machine-learning classifier. The reported integrated evaluation metrics are drawn from the shared project test environment and are interpreted in this thesis only where they directly support the individual component.

1.8 Individual contribution

The individual contribution is the design and integration of the driver/owner mobile experience around real-time telematics and operational reporting. The contribution includes the mobile interaction model, operational reporting workflows, driver-alert presentation, owner-side behaviour and trip visibility, integration with backend APIs, and evaluation of driver-event detection results relevant to the mobile component. The component transforms raw IoT telemetry into actionable feedback for the driver and understandable operational information for the bus owner.

1.9 Thesis structure

Chapter 2 reviews related work on IoT transport monitoring, smartphone and IMU-based driver behaviour analysis, and mobile operational systems. Chapter 3 explains the research methodology, data, procedures, metrics, ethics, and reproducibility approach. Chapter 4 presents the product and solution design. Chapter 5 documents implementation, integration, and testing. Chapter 6 presents results and validation. Chapter 7 discusses the findings, contributions, limitations, and implications. Chapter 8 covers commercialisation and sustainability. Chapter 9 concludes the thesis and proposes future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The literature review was organised around technologies directly relevant to the individual component: IoT transport telemetry, GPS and inertial sensing, mobile driver monitoring, real-time event detection, mobile feedback, and operational reporting. The review also considered the role of the component within a wider intelligent bus system so that the individual contribution could be clearly separated from route compliance, scheduling, and revenue functions owned by other group members.

The principal sources include peer-reviewed work on IoT architectures [1], operational bus telemetry [2], smartphone-based driver behaviour profiling [3], driving analytics [4], event-classification sensitivity [5], and systematic reviews of sensor-based driver behaviour research [6]. The project proposal and shared system thesis were also used to align the literature interpretation with the implemented architecture and the Sri Lankan public-bus context.

2.2 Domain and theoretical foundations

IoT-based transport systems combine sensing, communication, processing, and application layers. At the sensing layer, GPS provides spatial coordinates, velocity, and time information, while an IMU provides linear acceleration and angular velocity. At the communication layer, protocols such as MQTT and HTTPS transmit compact telemetry to cloud services. At the processing layer, backend services evaluate rules, persist events, and expose application programming interfaces. At the application layer, information is translated into user-facing alerts and reports [1].

Driver telematics treats driving behaviour as a pattern that can be inferred from motion and speed measurements. Research has shown that accelerometers, gyroscopes, GPS, and smartphones can identify behaviours such as harsh braking, rapid acceleration, cornering, and speeding [3]–[6]. For a public-bus context, a useful monitoring mechanism must also account for operational context. Low speed, for example, should not automatically be treated as unsafe because buses legitimately

stop at bus halts, traffic signals, and congestion points. Therefore, the proposed component uses duration and context conditions together with sensor thresholds.

2.3 Related research

Ferreira et al. investigated driver behaviour profiling using multiple smartphone sensors and machine-learning approaches, demonstrating the feasibility of classifying driving characteristics from mobile sensing data [3]. Vlahogianni and Barmpounakis reviewed smartphone driving analytics and highlighted the potential of mobile sensing together with challenges such as device placement, sensor noise, context dependence, and comparability between studies [4]. More recent work by Sarteshnizi and Tavakkoli Khomeini examined the sensitivity of event classification to classifier choice, sensor combinations, and acquisition rates [5].

A systematic review by Al-Hussein et al. grouped sensor-based driver-behaviour studies according to sensing technologies, motivations, methods, and challenges [6]. These studies establish that motion sensors are useful for safety monitoring, but much of the literature treats driver monitoring as a standalone analytical problem. The individual component in this thesis extends the practical context by linking detected events to a driver-facing mobile feedback workflow and bus-owner operational reports.

Bus-monitoring research also demonstrates the value of IoT and GPS for real-time operational visibility. Hounsell et al. showed how large-scale bus-fleet data can support operational management and planning [2]. The project proposal additionally reviewed IoT bus-monitoring systems that combine GPS and other onboard sensors. However, those systems generally concentrate on tracking or vehicle-condition monitoring rather than on a dual-role mobile application for driver reporting and owner visibility.

2.4 Existing systems products and techniques

Existing solutions relevant to this work can be grouped into four categories. First, Automatic Vehicle Location and passenger information solutions display live vehicle positions and estimated arrival information. Second, fleet telematics platforms capture speeding or harsh manoeuvres and provide safety scores, often

through fleet-management dashboards. Third, general mobile reporting applications allow field workers to submit incidents or status updates. Fourth, smart transport research prototypes combine IoT sensing with cloud dashboards. Each category provides useful capabilities, but the functions are usually separated across different products or stakeholder interfaces.

For the proposed bus setting, the component requires a single operational flow that begins with a driver using the app before a trip, continues with automated safety feedback during the trip, supports breakdown reporting if a disruption occurs, and ends with an operational summary that is visible to the owner. This end-to-end workflow is the key product distinction of the individual component.

2.5 Gap synthesis and conceptual framework

Table 2.1 Literature gap synthesis

Dimension	Typical prior approach	Gap addressed by this component
Vehicle monitoring	GPS location and ETA display	Uses IoT telemetry as input to driver safety feedback and operational reporting.
Driver telematics	Standalone safety scoring or post-trip analysis	Real-time in-trip alerts plus persistent event logs for the owner.
Operational reporting	Separate manual/phone-based status communication	Mobile bus availability, trip status, and breakdown reporting integrated with backend records.
Stakeholder visibility	Separate dashboards for tracking or fleet safety	Driver and owner roles within one mobile application with role-specific information.
Financial/operational context	Safety data isolated from broader management data	Owner can access trip summaries, schedules, earnings and route-related payment information from the integrated platform.

The conceptual framework therefore connects Sensor Input → Behaviour Analysis → Event Detection → Driver Feedback → Event Logging → Owner/Operational Visibility. This framework keeps the individual research focused on transforming telemetry into human-facing action while relying on other group components for route policy, scheduling, and revenue computation.

3 RESEARCH METHODOLOGY

3.1 Research design

This study follows Design Science Research (DSR), an approach suited to information-systems research in which an artefact is designed and evaluated to solve a practical problem [7]. The research activities followed six iterative stages: problem identification, definition of objectives, design and development, demonstration, evaluation, and communication. The artefact is the IoT-integrated driver monitoring and operational reporting mobile component together with its backend integration and driver-behaviour rules.

An iterative engineering process was used. Early versions of the mobile workflow were defined from operational requirements, connected to simulated telemetry, and later integrated with the shared IoT device and backend. Detection rules and operational flows were refined based on controlled driving scenarios and integration tests. This process allowed the research to evaluate both the analytical behaviour-detection mechanism and the usefulness of the mobile artefact as part of the complete system.

3.2 Data participants or study materials

Table 3.1 Data items and sources

Data / material	Source	Purpose
GPS latitude, longitude, speed, timestamp	NEO-6M GPS / shared IoT device	Vehicle movement context, overspeed detection, slow-movement context.
3-axis acceleration	MPU6050 IMU	Detect harsh braking and acceleration-related motion.
3-axis angular velocity	MPU6050 gyroscope	Detect aggressive turning / high yaw-rate events.
Operational app inputs	Driver mobile app	Bus availability, trip start/end, trip status, breakdown report.
Integrated platform data	Shared backend modules	Schedules, earnings and route-related payment details displayed to owners/drivers where relevant.
Stakeholder insights	RPTA consultations and project discussions	Clarify current operational practices and reporting requirements.

The driver-event evaluation used 120 controlled manoeuvre trials in the integrated project environment, comprising 30 samples for each of four categories: overspeeding, aggressive braking, aggressive turning, and abnormal slow movement. These shared empirical results directly support the evaluation of the individual behaviour-monitoring component.

3.3 Experimental or study procedure

1. Configure the onboard device to collect GPS and IMU telemetry during controlled or simulated bus-operation scenarios.
2. Transmit telemetry to the shared cloud/backend platform using secure MQTT/HTTPS communication.
3. Pre-process telemetry by aligning timestamps and filtering short-duration sensor noise where required.
4. Apply rule-based thresholds and duration conditions to classify driving events.
5. Generate an event record and driver-facing alert when a rule is satisfied.
6. Store the event in the backend so that it can be displayed in trip summaries and owner-side reports.
7. Verify operational mobile functions such as login, availability update, trip lifecycle updates, and breakdown reporting through API-based integration tests.
8. Compare detected driver events with controlled ground-truth labels to calculate detection accuracy.

3.4 Measures and evaluation metrics

Table 3.2 Evaluation measures

Metric	Definition / use
Driver-event detection accuracy	Percentage of controlled driving events correctly classified for each event category and overall.
Alert delivery / response path	Whether a detected event is converted into a mobile notification without breaking the end-to-end workflow.
Data transmission reliability	Consistency of telemetry and operational data transfer between device, backend, database, and mobile application.
Operational reporting correctness	Whether bus availability, trip status, breakdown, and summary information in the app matches backend records.
Client responsiveness	Integrated client rendering/refresh latency observed in the shared system benchmark.
System availability	Shared backend availability during continuous integrated testing.

3.5 Data analysis

For the driver-behaviour experiments, accuracy was calculated as the proportion of correctly detected labelled events within each event category. The overall telematics result was computed across all four categories. Descriptive comparison was then used to examine which event types were easier or more difficult to detect. Aggressive braking was expected to be more sensitive to road vibration and sensor noise, while GPS-based overspeeding was expected to be relatively stable when a reliable speed fix was available.

Operational functions were analysed through scenario-based verification. The expected application state and backend record were compared after each action. The integration analysis focused on continuity of the data path rather than on the internal algorithms of other group components.

3.6 Ethics privacy security and responsible AI

Vehicle telemetry and driver behaviour information can be privacy-sensitive. The research therefore uses anonymised identifiers rather than exposing unnecessary personal information. Monitoring is intended for active service periods and for safety/operational improvement rather than intrusive surveillance. Communication uses encrypted channels, while application access is controlled through authenticated roles. Driver and owner interfaces expose only information required for their roles.

Generative AI tools were used only for documentation assistance, language refinement, and code-debugging support in accordance with project disclosure requirements. Research decisions, design choices, implementation, test execution, interpretation, and final responsibility remain with the student and research team.

3.7 Reproducibility and research artefacts

The main reproducible artefacts are the mobile application source code, backend API definitions relevant to the mobile component, driver-behaviour rule configuration, controlled test cases, telemetry samples, and integration-test records. Version control through Git/GitHub was used to preserve code history. Postman was used for REST API verification, while Arduino serial tools were used during sensor

and firmware debugging. These artefacts enable future re-testing of event thresholds and mobile workflows.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Table 4.1 Functional requirements

ID	Requirement	Expected output
FR1	Authenticate driver or bus owner using assigned credentials.	Successful role-based login or clear error response.
FR2	Allow driver to update daily bus availability.	Availability state stored and visible to the integrated system.
FR3	Support trip start, trip status, and trip end updates.	Current trip lifecycle reflected in backend and app.
FR4	Allow driver to report a breakdown with trip/bus context.	Breakdown event stored and exposed to integrated operational modules.
FR5	Receive IoT sensor data and detect configured driver behaviours.	Event record with type, timestamp, and relevant sensor context.
FR6	Notify the driver immediately when a safety rule is triggered.	Mobile warning/alert corresponding to the detected event.
FR7	Allow bus owner to view trip summaries and driver-behaviour reports.	Role-specific owner reports and operational status.
FR8	Display schedule, earnings, and route-related payment information supplied by other integrated modules.	Consistent operational and financial visibility in the mobile app.

Table 4.2 Non-functional requirements

Category	Requirement
Performance	Alerts and operational updates should be processed with low latency suitable for real-time use.
Reliability	Temporary network interruptions should not corrupt trip or telemetry records; the shared device may buffer data for later transmission.
Security	Use encrypted transport, authenticated APIs, protected credentials, and role-based access.
Usability	Mobile screens should use clear labels, high-contrast status indicators, and simple driver interactions that minimise distraction.
Scalability	The service architecture should support

Category	Requirement
	increasing numbers of buses and concurrent telemetry streams.
Maintainability	Mobile, backend, and telemetry modules should be separated through APIs and modular services.

4.2 Component architecture within the complete solution

Figure 4.1 shows the individual component within the complete smart bus architecture. The IoT device layer collects physical vehicle telemetry. The cloud/communication layer forwards telemetry securely. The backend layer evaluates driver-behaviour rules and maintains operational records. The application layer contains two role-specific mobile experiences: the driver interface and the bus-owner interface.

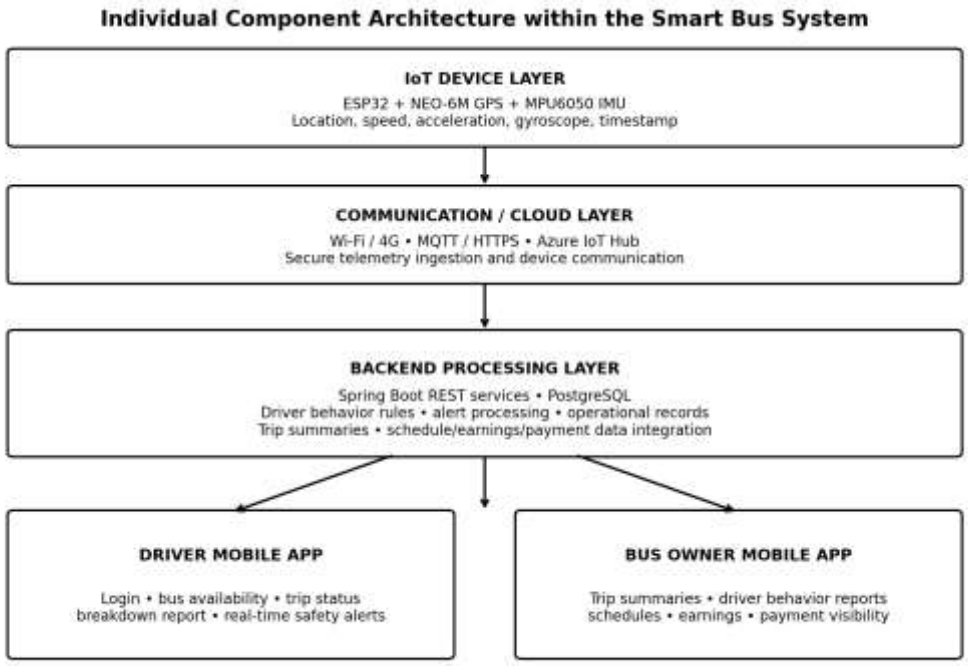


Figure 4.1 Component architecture within the complete solution

4.3 Design alternatives and decisions

Three important design alternatives were considered. First, smartphone-only sensing was considered because phones contain accelerometers, gyroscopes, and

GPS. The final design uses an onboard ESP32 with dedicated GPS and IMU sensors because the sensor position can be kept consistent and the telemetry can remain independent of the driver's personal phone. Second, a machine-learning classifier was considered for driving-event recognition. A rule-based approach was selected for the current prototype because its decisions are interpretable, can be tuned with limited labelled data, and are suitable for low-cost edge/cloud processing. Third, separate driver and owner applications were considered. A single React Native codebase with role-based interfaces was selected to reduce duplicated development while preserving different user permissions.

4.4 Detailed design

4.4.1 Driver behaviour detection design

The detection engine evaluates GPS speed together with IMU acceleration and angular velocity. The integrated implementation uses rule and duration conditions. Overspeeding is identified when GPS speed exceeds the configured road/operational threshold for a sustained period. Aggressive braking is detected from strong forward deceleration over a short window. Aggressive turning is identified from high yaw/angular-rate values while the vehicle is moving. Abnormal slow movement is treated as a contextual event requiring low speed for a sustained duration outside expected stop or traffic conditions.

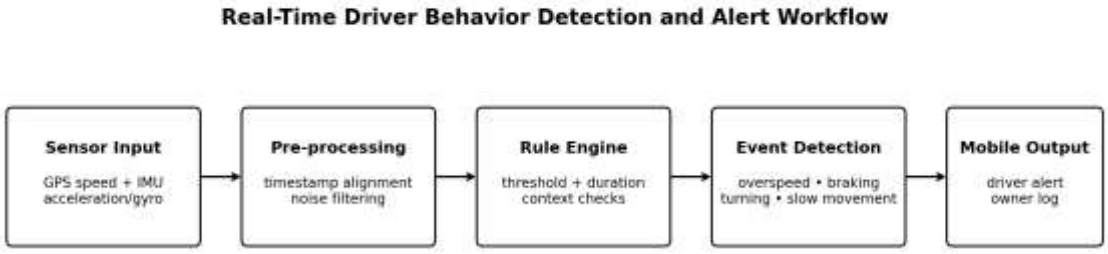


Figure 4.2 Real-time driver behaviour detection and alert workflow

4.4.2 Mobile interaction design

The driver interface prioritises a small set of operational actions that can be completed with minimal interaction: login, availability update, trip start/status/end, breakdown reporting, and alert acknowledgement. Safety alerts use short language and a clear event label so that the driver can understand the issue without reading a long message. The owner interface places emphasis on summaries rather than live interaction, presenting trip history, behaviour events, schedules, earnings, and route-related payment information.

4.4.3 Data model and API interaction

The mobile component communicates with Spring Boot REST services. Core logical entities include User, Driver, Bus, Trip, AvailabilityStatus, BreakdownEvent, DriverEvent, TripSummary, ScheduleView, EarningsView, and PaymentView. Telemetry is not continuously rendered as raw sensor data in the mobile application; instead, the backend converts sensor streams into meaningful events and summary data. This reduces interface complexity and limits unnecessary data exposure.

4.5 Standards and professional practice

The design follows common REST API principles, secure transport using TLS, JWT-based authentication, role-based access, source-code version control, and modular client/server separation. The user interface follows mobile usability principles by keeping driver interactions short and by avoiding dense dashboards during active service. Event logs maintain timestamps and traceability so that detected behaviours can be reviewed rather than relying only on transient alerts.

Table 4.3 Technology stack and justification

Technology	Selected option	Justification
Onboard controller	ESP32	Low-cost processing and wireless connectivity suitable for telemetry prototypes.
GPS	NEO-6M	Provides position, speed, and timestamps for movement context.
IMU	MPU6050	Provides 3-axis acceleration and 3-axis gyroscope data for driving-event detection.
Cloud/IoT	Azure IoT Hub	Supports secure device communication and scalable

Technology	Selected option	Justification
		telemetry ingestion.
Backend	Java Spring Boot	REST services, security integration, modular business logic.
Database	PostgreSQL	Reliable structured storage for operational and event records.
Mobile	React Native	Single cross-platform codebase for driver and owner interfaces.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The mobile application was designed around React Native, while the shared backend uses Java Spring Boot and PostgreSQL. The IoT device uses an ESP32 with NEO-6M GPS and MPU6050 IMU modules. Azure IoT Hub provides the cloud device-communication layer. Git/GitHub is used for version control, Postman for API testing, and Arduino IDE/Serial Monitor for sensor and firmware debugging. The separation between the mobile client and backend allows the same telemetry and operational records to be reused by other stakeholder interfaces in the complete system.

5.2 Component implementation

5.2.1 Operational mobile module

The operational module implements authentication, bus availability updates, trip lifecycle actions, and breakdown reporting. When the driver begins service, the application retrieves the relevant assignment and current operational state from the backend. Availability and trip updates are sent as REST requests. Breakdown reports create a structured incident record containing the bus/trip reference and timestamp so that the wider scheduling component can react to the disruption.

5.2.2 Driver telematics and alert module

Telemetry from the IoT device is processed by the shared backend/telematics pipeline. The individual component consumes the resulting driver events. Each event includes a category, timestamp, bus/trip context, and the values needed for

verification. Once an event is classified, the application receives or fetches the alert and displays a concise warning. The same event is retained for owner-side reporting and trip summaries.

Table 5.1 Driver behaviour rule configuration

Event	Core rule used in integrated prototype	Mobile response
Overspeeding	GPS speed above configured limit for a sustained interval (integrated test example: 5 s).	Show overspeed warning and log event.
Aggressive braking	Strong forward deceleration (integrated test threshold: > 3.5 m/s^2).	Show harsh-braking warning and log event.
Aggressive turning	High yaw rate (integrated test threshold: $\|\omega z\| > 25°/s$ while moving > 20 km/h).	Show aggressive-turn warning and log event.
Abnormal slow movement	Speed below 5 km/h for ≥ 60 s outside expected stop/traffic context.	Show slow-movement warning if context rule is satisfied and log event.

5.2.3 Owner reporting module

The owner role focuses on post-trip and operational visibility. The application displays trip summaries and driver-event information to help the owner understand whether the vehicle was operated safely. It also consumes schedule, earnings, and route-related payment data supplied by the integrated scheduling and revenue components. The mobile module does not recalculate those values; it presents them through a consistent owner-facing interface.

5.3 Integration with the other components

The individual component depends on clear service boundaries with the other group components. The IoT/route component supplies trusted device telemetry and route context. The policy/time-slot component supplies schedule and disruption outcomes. The revenue component supplies earnings and route-related payment results. The individual mobile component consumes these outputs and adds driver-side operational input, real-time safety feedback, and owner-facing presentation.

This design avoids duplicating algorithms. For example, a driver breakdown report is created in the mobile application, but any resulting standby-bus or schedule

adjustment is performed by the scheduling component. Similarly, earnings and payment amounts are displayed to the owner, but the calculation is owned by the revenue component.

5.4 Verification and software testing

Table 5.2 Functional verification scenarios

Scenario	Expected behaviour	Verification focus
Valid login	Correct role and dashboard are loaded.	Authentication and role routing.
Bus availability update	Availability changes are persisted and returned by the API.	Data consistency.
Trip start/end	Trip status changes and timestamps are stored.	Lifecycle correctness.
Breakdown report	Incident is stored and exposed to integrated scheduling services.	Integration event creation.
Overspeed event	Driver receives warning and owner log is created.	Detection-to-alert path.
Aggressive braking event	Driver receives warning and event remains visible in reports.	IMU event processing.
Owner summary view	Owner sees trip and driver-event history.	Role-specific reporting.
Schedule/earnings/payment view	Integrated values are displayed without client-side recalculation.	Cross-component consistency.

5.5 Deployment and operation

In the prototype deployment, the onboard device communicates over wireless/cellular connectivity to the cloud platform, while the mobile application accesses backend APIs over secure network connections. The mobile application is intended for Android/iOS deployment through React Native. During temporary connectivity loss, the system design relies on device/backend buffering and later synchronisation where supported by the shared platform. Operational logs are stored centrally so that the driver and owner can see consistent records after reconnection.

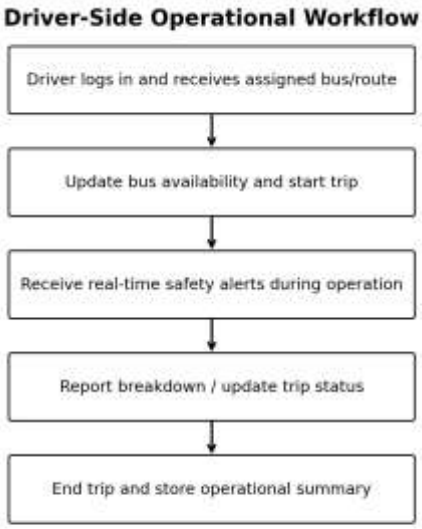


Figure 5.1 Driver-side operational workflow

6 RESULTS AND VALIDATION

6.1 Research results

The most direct quantitative result for the individual research component is the controlled evaluation of driver-behaviour detection. The integrated project reported 120 labelled manoeuvre trials, with 30 trials for each of the four event categories. The overall event-detection accuracy was 92.6%, exceeding the individual proposal target of at least 90% overall accuracy.

Table 6.1 Driver behaviour detection results

Driving event	Samples	Detection accuracy	Primary signal
Overspeeding	30	93.3%	GPS speed and duration
Aggressive braking	30	91.6%	IMU deceleration
Aggressive turning	30	91.8%	IMU yaw/angular rate with speed
Abnormal slow movement	30	93.6%	GPS speed, duration and context
Overall	120	92.6%	Integrated GPS + IMU rule set

The strongest result was abnormal slow movement at 93.6%, followed closely by overspeeding at 93.3%. Aggressive braking produced the lowest accuracy at 91.6%. This is consistent with the known sensitivity of low-cost accelerometers to engine vibration, road roughness, and device mounting, and it supports the need for filtering and calibration in practical deployments.

6.2 Product and technical results

The mobile component was integrated with the shared platform so that operational records and driver-event outputs could be consumed through role-specific interfaces. The wider integrated benchmark reported a client web/mobile dashboard render and status-update time of approximately 1.4 seconds. The shared backend also achieved 99.4% availability in a 72-hour load test. These values are system-level results rather than measurements produced solely by the mobile component, but they provide evidence that the infrastructure supporting the app can respond at a practical operational speed.

The core mobile use cases—authentication, bus availability update, trip state update, breakdown reporting, driver alert presentation, and owner summary viewing—were verified through scenario-based integration tests. Schedule, earnings, and payment information are consumed from the relevant group modules and displayed by the mobile client; their calculation correctness is outside the individual component's ownership.

6.3 User or stakeholder validation

The application requirements were derived from stakeholder discussions and the operational context provided by the Road Passenger Transport Authority. The resulting role separation reflects the practical needs identified in the proposal: the driver requires a simple operational interface and immediate alerts, while the owner requires summaries and visibility rather than continuous raw telemetry. A formal large-sample usability study or final System Usability Scale score was not available in the supplied project evidence; therefore, this thesis does not claim a quantitative usability score. Usability is discussed based on requirement fit and functional verification.

6.4 Integration results

The integration tests confirmed the intended separation of responsibilities. Telemetry originates from the onboard device, behaviour events are produced by the processing layer, and the mobile application delivers alerts and reporting. Breakdown events created in the driver app can be consumed by the scheduling component. Owner-facing schedule and financial values can be retrieved from other modules without recreating their algorithms in the client. This modular integration supports the overall project goal while preserving the originality of each group member's research component.

6.5 Summary of findings

Table 6.2 Results mapped to individual objectives

Objective	Evidence / result	Status
Develop driver/owner mobile operational workflow	Authentication, availability, trip, breakdown and summary scenarios integrated with backend APIs.	Achieved in prototype
Integrate IoT sensor data	GPS and 6-axis IMU data available through shared IoT/cloud pipeline.	Achieved in integrated system
Detect four driver behaviours	92.6% overall accuracy across 120 controlled trials.	Target achieved
Generate real-time driver alerts and retain logs	Detection-to-alert and owner-log flow implemented.	Achieved in prototype
Provide owner operational/financial visibility	Trip/behaviour summaries plus schedule, earnings and payment views consume integrated module outputs.	Achieved at integration level

7 DISCUSSION

7.1 Interpretation of findings

The results indicate that low-cost GPS and IMU sensing is sufficiently accurate for a practical rule-based driver-feedback prototype. An overall accuracy of 92.6% demonstrates that the four target event types can be distinguished with useful reliability in controlled testing. The result is particularly important because the purpose of the component is not only to record events after a trip; it is to convert them into timely feedback that can influence driver awareness during operation.

The component also demonstrates the value of joining automated telemetry with driver-entered operational information. Sensor data alone cannot communicate that a bus is intentionally unavailable or has experienced a breakdown. Conversely, manual operational reporting alone cannot objectively detect harsh braking or aggressive turning. Combining both information types within the same mobile workflow creates a more complete operational picture.

7.2 Comparison with literature baselines and existing solutions

Table 7.1 Comparison with prior approaches

Research focus	Prior literature / common approach	Individual component outcome
Driver behaviour profiling	Smartphone or sensor analytics classify driving patterns [3]–[6].	Dedicated onboard GPS/IMU telemetry is connected to real-time driver alerts and owner reports.
Bus operational monitoring	AVL/fleet systems emphasise location and fleet visibility [2].	Adds driver-entered availability, trip status and breakdown reporting.
User feedback	Many telematics studies focus on classification accuracy or post-trip scoring.	Provides in-trip mobile warnings with persistent event logs.
Stakeholder integration	Separate safety and operations dashboards are common.	Driver and owner roles share one mobile product integrated with scheduling and revenue outputs.

The novelty should therefore be understood as an integration contribution rather than a claim that overspeeding or harsh-braking detection is new by itself. The research contribution lies in combining sensor-based driver monitoring, immediate feedback, and bus-specific operational reporting in a role-based mobile application connected to a broader smart-bus management system.

7.3 Research contribution

The research contribution is a component-level framework for translating real-time bus telemetry into actionable mobile safety feedback and operational evidence. It demonstrates how driver-behaviour rules can be embedded in an IoT transport workflow and how event records can be reused for owner-side monitoring. The work

also clarifies the boundary between mobile presentation and the algorithms owned by other modules, supporting a clean multi-component research architecture.

7.4 Product contribution

The product contribution is a React Native driver/owner mobile application integrated with a Spring Boot backend and shared IoT platform. The application supports operational communication before, during, and after a trip, while exposing driver-safety information without requiring the user to interpret raw sensor streams. The owner interface provides a compact operational view that brings together trip information, behaviour events, schedules, earnings, and payment-related information.

7.5 Limitations and threats to validity

- The 120 driving-event trials are controlled tests and may not represent the full diversity of road, weather, vehicle, and traffic conditions in nationwide public-bus operation.
- MPU6050 readings can be affected by engine vibration, rough roads, and sensor mounting, particularly for aggressive-braking classification.
- Abnormal slow movement requires contextual filtering because legitimate bus stops and congestion can produce low-speed patterns similar to inefficient operation.
- The current approach uses rules and thresholds rather than a personalised or machine-learning model, so fixed thresholds may not generalise equally to all vehicle types and routes.
- The supplied project evidence does not include a large-sample formal mobile usability study; usability conclusions are therefore limited to requirement fit and functional testing.
- Schedule, earnings, and payment values shown in the app depend on other integrated group components and are not independently computed by this individual component.

7.6 Implications

For bus drivers, the component can increase awareness of unsafe manoeuvres through immediate feedback. For bus owners, it can provide evidence-based visibility into trips and driver behaviour rather than relying only on verbal reports. For transport authorities, the component can improve the quality of operational information available to the wider platform. At a research level, the work shows that

safety monitoring becomes more useful when integrated with the operational workflow in which drivers and owners already make decisions.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The value proposition is a low-cost mobile and IoT service that helps bus operators combine daily operational reporting with driver-safety monitoring. Primary users are bus drivers and private bus owners. Institutional customers include Road Passenger Transport Authorities, the National Transport Commission, and bus-operator associations that require reliable operational evidence across fleets.

8.2 Market and competitive position

Commercial GPS trackers, fleet telematics products, and generic incident-reporting applications already exist. The proposed component is positioned differently by combining bus-specific operational workflows with real-time driver feedback and by integrating with scheduling and distance-based payment functions of the wider research platform. This reduces the need for separate systems for driver reporting, safety events, and operator summaries.

8.3 Business or sustainability model

Table 8.1 Commercialisation model

Element	Proposed model
Customer	Transport authorities, bus operator associations, private fleet owners.
Core service	Driver/owner mobile app, IoT telemetry integration, behaviour alerts, operational summaries.
Revenue	Platform subscription, per-bus monthly monitoring fee, hardware setup or leasing fee, maintenance/support.
Cost drivers	IoT hardware, cellular data, cloud hosting, technical support, security monitoring.
Sustainability	Shared cloud infrastructure and one cross-platform mobile codebase reduce marginal cost as fleet size grows.

8.4 Scalability intellectual property and compliance

The modular API-based architecture allows additional buses and users to be added without redesigning the mobile client. Intellectual property value lies primarily in the integrated workflow, event-rule configuration, mobile interaction design, and system integration know-how. Commercial deployment would need to comply with applicable data-protection, transport-regulation, cybersecurity, and employment/privacy requirements. Driver-monitoring policies should be transparent, proportionate, and limited to legitimate safety and operational purposes.

8.5 Contribution to the Sustainable Development Goals

The work supports Sustainable Development Goal 11 by contributing to safer and more reliable urban transport, and it supports Goal 9 by applying IoT and digital infrastructure to transport operations. The driver-safety component also contributes indirectly to Goal 3 by supporting safer road travel. These contributions depend on responsible deployment, accurate monitoring, and fair use of driver data.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the design, implementation, and evaluation of an IoT-Integrated Driver Monitoring and Operational Reporting Mobile Application for a smart bus management system. The component addresses a practical gap between bus operational reporting and sensor-based driver monitoring. It enables drivers to report availability, trip status, and breakdowns while receiving real-time warnings generated from GPS and IMU telemetry. It also provides bus owners with trip summaries, driver-behaviour information, schedules, earnings, and route-related payment visibility through the integrated platform.

The integrated driver-behaviour evaluation achieved 92.6% overall detection accuracy across 120 controlled trials, with all four event categories exceeding 91% accuracy. These results indicate that a low-cost ESP32, GPS, IMU, cloud, backend, and mobile architecture can provide useful real-time safety feedback. The

component's main contribution is the integration of driver safety monitoring with everyday operational reporting in a single role-based mobile workflow.

9.2 Achievement of objectives

The mobile operational workflow was designed and integrated; IoT telemetry was connected through the shared system; four target driver behaviours were implemented and evaluated; alerts and persistent event logs were provided; and owner-side operational visibility was integrated. The primary quantitative behaviour-detection target was achieved, while the usability evaluation remains an area for stronger future empirical work.

9.3 Individual research and product contributions

- A bus-specific mobile workflow linking driver operational reporting and real-time telematics feedback.
- A rule-based driver behaviour mechanism for overspeeding, aggressive braking, aggressive turning, and abnormal slow movement using GPS and IMU inputs.
- A driver alert and event-log pipeline that transforms raw telemetry into actionable feedback and owner-visible evidence.
- A role-based React Native interface for drivers and bus owners integrated with the wider smart-bus scheduling and revenue ecosystem.
- Empirical validation of the driver-event mechanism using 120 controlled trials with 92.6% overall accuracy.

9.4 Future directions

- Evaluate the mobile application with a larger sample of active bus drivers and owners using formal usability instruments such as the System Usability Scale.
- Introduce adaptive or machine-learning thresholds that account for vehicle type, route geometry, road surface, and individual driving patterns.
- Improve context recognition for abnormal slow movement by integrating bus-stop geofences, congestion data, and schedule context.
- Add offline-first mobile synchronisation for areas with weak cellular coverage.
- Investigate edge AI / TinyML on the ESP32 so that selected driver events can be detected locally with reduced cloud dependence.
- Extend owner analytics with longitudinal safety trends and coaching recommendations while maintaining transparent and privacy-preserving use of driver data.

REFERENCES

[1] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," IEEE Communications Surveys & Tutorials, vol. 17, no. 4, pp. 2347–2376, 2015.

[2] N. B. Hounsell, B. P. Shrestha, and A. Wong, "Data management and applications in a world-leading bus fleet," Transportation Research Part C: Emerging Technologies, vol. 22, pp. 76–87, 2012.

[3] J. Ferreira Júnior, E. Carvalho, B. V. Ferreira, C. de Souza, Y. Suhara, A. Pentland, and G. Pessin, "Driver behavior profiling: An investigation with different smartphone sensors and machine learning," PLOS ONE, vol. 12, no. 4, e0174959, 2017.

[4] E. I. Vlahogianni and E. N. Barmpounakis, "Driving analytics using smartphones: Algorithms, comparisons and challenges," Transportation Research Part C: Emerging Technologies, vol. 79, pp. 196–206, 2017.

[5] I. T. Sarteshnizi and F. Tavakkoli Khomeini, "Sensitivity analysis of driving event classification using smartphone motion data: Case of classifier type, sensor bundling, and data acquisition rate," Journal of Intelligent Transportation Systems, vol. 28, no. 4, pp. 476–493, 2024.

[6] W. A. Al-Hussein, M. L. Kiah, and P. L. Yee, "A systematic review on sensor-based driver behaviour studies: Coherent taxonomy, motivations, challenges, recommendations, substantial analysis and future directions," PeerJ Computer Science, vol. 7, e632, 2021.

[7] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Quarterly, vol. 28, no. 1, pp. 75–105, 2004.

[8] S. D. Sarathkumar, R. A. Raj, and M. Suresh, "IoT Based Intelligent Bus Monitoring System," in 2023 International Conference on Electrical, Electronics, Communication and Computers (ELEXCOM), Tamil Nadu, India, 2023.

[9] M. A. Quddus, W. Y. Ochieng, and R. B. Noland, "Current map-matching algorithms for transport applications: State-of-the-art and future research directions," Transportation Research Part C: Emerging Technologies, vol. 15, no. 5, pp. 312–328, 2007.

[10] S. J. Berrebi, E. Hans, N. Chiabaut, J. A. Laval, L. Leclercq, and K. E. Watkins, "Comparing bus holding methods with and without real-time predictions," Transportation Research Part C: Emerging Technologies, vol. 87, pp. 197–211, 2018.

[11] National Transport Commission, Transport Statistics / Private Bus Operational Data, Ministry of Transport, Sri Lanka, 2022.

GLOSSARY

Term	Definition
Driver behaviour event	A classified driving pattern such as overspeeding, aggressive braking, aggressive turning, or abnormal slow movement.
Operational reporting	Driver-entered information about bus availability, trip status, or breakdowns that supports management decisions.
Telematics	The combined collection, transmission, and analysis of vehicle location and motion data.

Term	Definition
Rule-based detection	Classification using explicit thresholds, durations, and context rules rather than a learned statistical model.
Owner visibility	Role-specific access to trip summaries, driver behaviour reports, schedules, earnings, and payment information.

APPENDICES

Appendix A Research instruments or extended results

Suggested evidence to attach before final submission includes the controlled driver-event test sheet, sample telemetry records, API test cases, and anonymised stakeholder interview/questionnaire material. The thesis body reports only results supported by the supplied project evidence.

Appendix A.1 Example driver-event validation record

Field	Example content
Trial ID	DB-001
Event label	Aggressive Braking
Ground truth	Controlled braking manoeuvre
Sensor features	GPS speed, ax, ay, az, ωx, ωy, ωz, timestamp
System output	Aggressive Braking event + mobile alert
Verification	Detected / Not detected

Appendix B Academic integrity and AI use disclosure

AI-assisted tools, including ChatGPT and Claude, were used for language refinement, documentation support, and code-debugging assistance. They were not treated as sources of empirical results. All technical designs, requirements, algorithms, source-code changes, test execution, research decisions, and reported results remain the responsibility of the student and research team. AI-generated suggestions were reviewed and modified before inclusion.